The State-Action Value Function ($Q(s, a)$) 💎

To solve the Reinforcement Learning problem, we need a specific quantity that tells us *how good* a particular action is. This is the **Q-Function**.

Definition

$Q(s, a)$ is a function that takes a State s and an Action a and returns a single number.

$Q(s, a) = \text{The Return if you start in state } s, \text{ take action } a \text{ once, and then behave optimally forever after.}$

- **Ideally:** This number represents the *best possible total reward* you can get after taking that specific action.
- **Notation:** $Q(s, a)$ stands for the **"Quality"** of taking action a in state s.

The "Circular" Logic

- **The Catch:** The definition says "behave optimally after." But we are trying to *find* the optimal behavior! How can we compute Q if we don't know the optimal policy yet?
- **The Solution:** Don't worry about this for now. RL algorithms (like Q-Learning) have clever ways to estimate this value iteratively without knowing the optimal policy in advance.

Example Calculation (Mars Rover)

Let's assume we know the optimal policy for the Mars Rover (Go Left at 2, 3, 4; Go Right at 5). Let's calculate Q for specific state-action pairs. ($\gamma = 0.5$).

1. Calculate $Q(s_2, \text{Right})$

- **Scenario:** You are at State 2. You force the robot to go **Right** (a bad move), but then let it act optimally (go Left) afterward.
- **Path:** Start at $2 \xrightarrow{\text{Right}} 3 \xrightarrow{\text{Left}} 2 \xrightarrow{\text{Left}} 1$ (Goal).
- **Rewards:** 0 (at 2) $\rightarrow$ 0 (at 3) $\rightarrow$ 0 (at 2) $\rightarrow$ 100 (at 1).
- **Return:** $0 + 0.5(0) + 0.5^2(0) + 0.5^3(100) = 12.5$.
- **Result:** $Q(s_2, \text{Right}) = 12.5$.

2. Calculate $Q(s_2, \text{Left})$

- **Scenario:** You are at State 2. You force the robot to go **Left** (a good move), and let it act optimally afterward.
- **Path:** Start at $2 \xrightarrow{\text{Left}} 1$ (Goal).
- **Rewards:** 0 (at 2) $\rightarrow$ 100 (at 1).
- **Return:** $0 + 0.5(100) = 50$.
- **Result:** $Q(s_2, \text{Left}) = 50$.

Using $Q(s, a)$ to Pick Actions

Once we have calculated $Q(s, a)$ for every possible action in a state, picking the best action is incredibly simple: **Just pick the highest number.**

- **At State 4:**
  - $Q(s_4, \text{Left}) = 12.5$
  - $Q(s_4, \text{Right}) = 10$
  - **Decision:** $12.5 > 10$, so the optimal action is **Left**.
- **At State 5:**
  - $Q(s_5, \text{Left}) = 6.25$
  - $Q(s_5, \text{Right}) = 20$
  - **Decision:** $20 > 6.25$, so the optimal action is **Right**.

The Optimal Policy Formula

The optimal policy $\pi(s)$ is simply the action that maximizes the Q-value.

$\pi(s) = \max_a Q(s, a)$

- This is why computing the Q-function is so important. If we know $Q(s, a)$, we automatically know the optimal policy!

Building Intuition: Changing Rewards and Discount Factors 🎛️

The best way to understand how Reinforcement Learning works is to see how the agent's behavior (Policy) changes when you tweak the "rules of the world." In the optional lab, we modify the **Mars Rover** example to see how $Q(s, a)$ adapts.

1. The Baseline Scenario

- **Left Reward (State 1):** +100
- **Right Reward (State 6):** +40
- **Discount Factor (γ):** 0.5
- **Original Policy:** Go Left from states 2, 3, 4. Go Right from state 5.

Experiment 1: Changing the Reward

Scenario: What if we drastically reduce the reward on the right side?

- **Change:** We lower the Right Reward from **+40** to **+10**.
- **Effect on $Q(s, a)$:**
  - Previously, at State 5, going Right was worth it (Reward 40 vs. traveling far for 100).
  - Now, going Right yields very little value. Even though the big reward (+100) is far away, it is mathematically worth the trip compared to the measly +10 nearby.
- **New Policy:** The robot now goes **Left** from *every* state, including State 5.

Experiment 2: Increasing Patience (γ)

Scenario: What if we make the robot more patient and forward-thinking?

- **Change:** We increase the Discount Factor (γ) from 0.5 to **0.9**.
- **Effect on $Q(s, a)$:**
  - $\gamma = 0.9$ means future rewards retain most of their value (0.9, 0.81, 0.72...).
  - The robot is no longer "afraid" of the distance to the big reward.
  - At State 5, the "return" for traveling all the way left to the +100 reward becomes higher than grabbing the immediate +40.
- **New Policy:** The robot goes **Left** from State 5 (and all other states).

Experiment 3: Increasing Impatience (γ)

Scenario: What if we make the robot extremely short-sighted?

- **Change:** We decrease the Discount Factor (γ) from 0.5 to **0.3**.
- **Effect on $Q(s, a)$:**
  - $\gamma = 0.3$ means future rewards lose value incredibly fast (0.3, 0.09, 0.027...).
  - Ideally, the robot wants +100. But from State 4, the +100 is "too far away" in terms of discount steps. The value decays to almost nothing before the robot gets there.
  - The +40 on the right is closer.
- **New Policy:** The robot now goes **Right** from State 4 (whereas it used to go Left). It settles for the smaller reward because it is closer.

Summary of Intuition

Adjustment	Resulting Behavior
Lower a Reward	The agent will likely avoid that path and travel further to find a better reward.
High γ (e.g., 0.99)	**Patient / Strategic.** The agent will ignore small nearby rewards to travel long distances for a massive payout.
Low γ (e.g., 0.1)	**Impatient / Greedy.** The agent cares only about what is right next to it. It will choose small nearby rewards over massive distant ones.

The Bellman Equation 🔔

We know that $Q(s, a)$ represents the *best possible return* after taking action a in state s. But how do we calculate this value? The **Bellman Equation** provides a recursive formula to compute $Q(s, a)$.

The Core Idea: Breaking Down the Return

The total return is a sum of discounted rewards. We can split this sum into two parts:

1. **Immediate Reward ($R(s)$):** The reward you get *right now* for being in the current state.
2. **Future Return:** The discounted value of everything that happens *after* the next step.

The Equation

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

- s: Current State.
- a: Current Action.
- s': Next State (where you land after taking action a).
- a': Possible actions in the next state.
- $R(s)$: Immediate Reward.
- γ: Discount Factor.
- $\max_{a'} Q(s', a')$: The best possible return you can get from the *next* state s'.

Intuition: "Current Reward + Best Future"

The equation says:
*"The value of taking an action now is equal to the **reward I get right now**, plus the **discounted value of the best possible situation** I will be in at the next step."*

Example Calculation (Mars Rover)

Let's verify this equation with our Mars Rover example ($\gamma = 0.5$).

Calculate $Q(s_4, \text{Left})$:

1. **Current State (s):** State 4.
2. **Action (a):** Left.
3. **Next State (s'):** State 3.
4. **Immediate Reward ($R(s_4)$):** 0.

5. **Best Future Value from State 3 ($\max Q(s_3, a')$):**
    - We know from previous calculations that at State 3, the returns are:
        - $Q(s_3, \text{Left}) = 25$
        - $Q(s_3, \text{Right}) = 6.25$
    - So, $\max(25, 6.25) = \mathbf{25}$.

Apply Bellman Equation:
$Q(s_4, \text{Left}) = R(s_4) + 0.5 \times \max Q(s_3, a')$
$Q(s_4, \text{Left}) = 0 + 0.5 \times 25$
$Q(s_4, \text{Left}) = \mathbf{12.5}$

This matches exactly with the manual calculation we did earlier using the full sum of rewards!

Why is this Useful?

Instead of having to sum up infinite future rewards (which is hard), the Bellman Equation allows us to compute $Q(s, a)$ using only the **immediate reward** and the **Q-values of the next state**. This recursive structure is what allows algorithms to "learn" the Q-values step by step.

Stochastic Reinforcement Learning 🎲

Up until now, we have assumed that the environment is **Deterministic**: if you command the robot to go Left, it *always* goes Left.
In the real world, outcomes are rarely 100% reliable. Floors might be slippery, wind might blow a drone off course, or wheels might slip. We call these **Stochastic Environments** (environments involving randomness).

The Mars Rover Example (Stochastic Version)

Imagine the same Mars Rover setup, but now the robot is imperfect.

- **Command:** Go Left.
  - **90% Chance (0.9):** Robot successfully moves **Left**.
  - **10% Chance (0.1):** Robot slips and accidentally moves **Right**.
- **Command:** Go Right.
  - **90% Chance (0.9):** Robot successfully moves **Right**.
  - **10% Chance (0.1):** Robot slips and accidentally moves **Left**.

The Challenge: Random Rewards

Because the movement is random, the sequence of states and rewards is no longer guaranteed. Even if you follow the exact same Policy, the outcome can change every time you run the robot.

- **Run 1 (Lucky):** You command Left. It goes Left (Reward 0) $\rightarrow$ Left (Reward 0) $\rightarrow$ Left (Reward 100). **High Return.**
- **Run 2 (Unlucky):** You command Left. It slips Right (Reward 0). You command Left. It slips Right again (Reward 0). It takes much longer to reach the goal. **Low Return.**

The New Goal: Expected Return

Since we cannot guarantee a specific return, we change our goal. We want to maximize the **Expected Return** (which is the statistical term for the **Average Return**).

If we ran the robot through the maze 1,000 times, we want the *average* sum of discounted rewards to be as high as possible.

Mathematical Notation:
Maximize $E[R_1 + \gamma R_2 + \gamma^2 R_3 + \dots]$

- $E[\dots]$: The Expected Value (Average) operator.

Modified Bellman Equation

The Bellman Equation must be updated to account for this randomness. Since the Next State (s') is not guaranteed, we must take the **average** of the possible future outcomes.

$Q(s, a) = R(s) + \gamma E[\max_{a'} Q(s', a')]$

- $R(s)$: The immediate reward (this is usually known).
- $E[...]$: We average the best possible future returns ($Q(s', a')$) based on the probability of landing in each possible next state s'.

Intuition on Control

- **Low Stochasticity (e.g., 1% slip):** You have high control. Your Q-values and Expected Returns will be high.
- **High Stochasticity (e.g., 40% slip):** You have low control. Your actions matter less because the environment is chaotic. Your Q-values and Expected Returns will decrease significantly.